CHAPTER 01

Core Java

Primitives · Collections · Generics · Concurrency · JVM

Chapter Overview

Chapter 01 lays the Java foundation that every ShopVerse class is built on. It spans 13 sub-chapters: primitive types, Collections (List/Set/Map/Queue), their internal implementations, Generics, exception hierarchy, Java 8 functional APIs, Java 9-21 modern features, multithreading, java.util.concurrent utilities, and JVM tuning flags.

Area	Key APIs / Classes	ShopVerse File
Primitives & Types	long, BigDecimal, char[]	MoneyConverter, JwtProperties
Collections – List	ArrayList, LinkedList, CopyOnWriteArrayList	OrderService, NotificationDispatcher, EventListenerRegistry
Collections – Set	HashSet, TreeSet, EnumSet, LinkedHashSet	ProductTag, CategoryService, OrderValidator
Collections – Map	LinkedHashMap, ConcurrentHashMap, EnumMap	ShoppingCart, RateLimiter, LocaleConfig
Collections – Queue	ArrayBlockingQueue, ArrayDeque, PriorityQueue	AsyncNotificationQueue, OrderProcessingPipeline
Collections Internals	HashMap bucket/tree, load factor, CAS	CartService (comment block)
Generics	ApiResponse<T>, PagedResponse<T>, Result<T,E>	shopverse.web.dto, domain.repository
Exceptions	ShopVerseException hierarchy, ErrorCode enum	shopverse.common.exception
Java 8	Stream, Optional, LocalDate, CompletableFuture	OrderService, NotificationService
Java 9-21	Records, Sealed, Pattern switch, Virtual threads	PlaceOrderRequest, OrderEvent, EventHandler
Multithreading	ExecutorService, ReentrantLock, Semaphore, Future	NotificationThreadPool, FlashSaleService
j.u.concurrent	AtomicLong, CountDownLatch, CopyOnWriteArrayList	OrderIdGenerator, StartupValidator
JVM Internals	G1GC, Metaspace, ManagementFactory, HeapDump	Dockerfile, JvmMetricsEndpoint

01-01 Java Primitives & Type System

Primitive Types Reference

Type	Bits	Min	Max	ShopVerse Rule
byte	8	-128	127	Avoid – sign extension traps in bit ops
short	16	-32 768	32 767	Not used in ShopVerse
int	32	~-2.1B	~2.1B	qty, pageNumber, version counter
long	64	~-9.2×10¹■	~9.2×10¹■	ALL entity IDs (order, product, customer)
float	32 (IEEE 754)	~±1.4×10■■■	~±3.4×10³■	NEVER for money
double	64 (IEEE 754)	~±5×10■³²■	~±1.8×10³■■	NEVER for money
boolean	1 (JVM uses int)	false	true	Feature flags, isActive, isFeatured
char	16 (UTF-16)	0	65535	char[] for JWT secret only

Why BigDecimal for Money?

```
// IEEE-754 binary float CANNOT represent 0.1 exactly
double a = 0.1, b = 0.2; System.out.println(a + b); // 0.30000000000000004 ← WRONG
BigDecimal x = new BigDecimal("0.1"), y = new BigDecimal("0.2");
System.out.println(x.add(y)); // 0.3 ← CORRECT
```

Operation	double Result	BigDecimal Result	Safe?
0.1 + 0.2	0.30000000000000004	0.3	BigDecimal ✓
1.03 - 0.42	0.6100000000000001	0.61	BigDecimal ✓
4.015 × 100	401.49999999999994	401.5	BigDecimal ✓
10.0 / 3	3.3333333333333335	3.333… (scale ctrl)	BigDecimal ✓

char[] vs String for Secrets

```
// BAD: String stays in pool until GC; heap dump exposes it
private String jwtSecret = "supersecret";
// GOOD: char[] can be zeroed after use
private char[] jwtSecret = env("JWT_SECRET").toCharArray();
// Cleanup on shutdown:
Arrays.fill(jwtSecret, '\0');
```

■ JwtProperties.java uses char[] specifically to prevent the JWT secret from appearing in heap dumps or being retained in the String constant pool.

Widening & Narrowing Casts

```
// MoneyConverter.java – explicit cast comment
long cents = (long)(doubleAmount * 100); // widening double→long: risk of precision loss
int qty = (int) longQty; // narrowing long→int: silently wraps at MAX_INT
// Rule: always document narrowing casts and validate range first
```

■ Use Math.toIntExact(long) to throw ArithmeticException on overflow instead of silent truncation.

List Implementation Comparison

Impl	Backing Store	get(i)	add(end)	add(mid)	remove(mid)	ShopVe
ArrayList	Object[]	O(1)	O(1) amort.	O(n)	O(n)	OrderItem list – O(1) inde
LinkedList	Doubly-linked nodes	O(n)	O(1)	O(1) ptr	O(1) ptr	NotificationDispatcher – C
CopyOnWriteArrayList	volatile Object[] (new copy on write)	O(1)	O(n)	O(n)	O(n)	EventListenerRegistry – r
Vector (legacy)	synchronized Object[]	O(1)	O(1) amort.	O(n)	O(n)	Do not use – prefer Copy

```
List items = new ArrayList<>(); // ordered, fast index
Queue q = new LinkedList<>(); // O(1) offer/poll
List reg = new CopyOnWriteArrayList<>(); // safe under concurrent reads
return Collections.unmodifiableList(new ArrayList<>(this.items)); // defensive copy
```

Iteration Patterns

```
// 1. Enhanced for – simplest, preferred for read-only iteration
for (OrderItem item : order.getItems()) { process(item); }
// 2. Iterator – safe to remove during iteration
Iterator it = cart.values().iterator();
while (it.hasNext()) { if (it.next().qty() == 0) it.remove(); }
// 3. ListIterator – bidirectional, supports set()
ListIterator li = list.listIterator(list.size());
while (li.hasPrevious()) process(li.previous());
```

01-03 Collections – Set

Set Implementation Comparison

Impl	Order	contains	add	Null?	ShopVerse Usage
HashSet	None	O(1) avg	O(1) avg	1 null	ProductTag – O(1) tag lookup
LinkedHashSet	Insertion order	O(1) avg	O(1) avg	1 null	When iteration order must match insert orde
TreeSet	Sorted (Comparable or Comparator)	O(log n)	O(log n)	No null	CategoryService – alphabetical iteration
EnumSet	Enum ordinal	O(1)	O(1)	No null	OrderValidator – CANCELLED/DELIVERED
CopyOnWriteArraySet	Insertion order	O(n)	O(n)	Allowed	Tiny concurrent sets; reads >> writes

```
Set tags = new HashSet<>(List.of("electronics","sale"));
Set sorted = new TreeSet<>(Comparator.comparing(Category::getName));
EnumSet terminal = EnumSet.of(DELIVERED, CANCELLED, REFUNDED);
if (terminal.contains(order.getStatus())) throw new InvalidTransitionException();
```

Set Operations

Operation	Method	Example Use
Union (A∪B)	Set.addAll(other)	Merge product tags from two collections
Intersection (A∩B)	Set.retainAll(other)	Products tagged BOTH "sale" AND "electronics"
Difference (A−B)	Set.removeAll(other)	Tags present in A but not B
Subset check	Set.containsAll(other)	Validate all required tags are present

01-04 Collections – Map

Map Implementation Comparison

Impl	Order	get/put	Null keys	Thread-safe	ShopVerse Usage
HashMap	None	O(1) avg	1 null	No	General purpose (not in concurrent coo
LinkedHashMap	Insertion order	O(1) avg	1 null	No	ShoppingCart – display preserves add
TreeMap	Key-sorted	O(log n)	No null	No	Price tiers, sorted config maps
ConcurrentHashMap	None	O(1) avg	No null	Yes (CAS)	RateLimiter<userId, AtomicLong>
EnumMap	Enum ordinal	O(1)	No null	No	LocaleConfig<SupportedLocale, Messa
WeakHashMap	None	O(1) avg	1 null	No	GC-reclaimable key caches
Collections.synchronizedMap	Same as wrapped	Same	Same	Yes (coarse)	Legacy; prefer ConcurrentHashMap

```
private final Map cart = new LinkedHashMap<>();
private final ConcurrentHashMap rate = new ConcurrentHashMap<>();
rate.computeIfAbsent(userId, k -> new AtomicLong()).incrementAndGet();
private final EnumMap bundles = new EnumMap<>(SupportedLocale.class);
```

computeIfAbsent / merge / getOrDefault Patterns

```
// computeIfAbsent – create only if absent (thread-safe in CHM)
counterMap.computeIfAbsent(key, k -> new AtomicLong(0)).incrementAndGet();
// merge – update or insert in one call
inventory.merge(productId, 1, Integer::sum); // increment stock
// getOrDefault – safe retrieval without null check
int qty = cart.getOrDefault(productId, CartItem.EMPTY).qty();
```

01-05 **Collections – Queue & Deque**

Queue & Deque Implementations

Class	Type	Blocking?	Bounded?	Order	ShopVerse Usage
ArrayBlockingQueue	Queue	Yes	Yes (500)	FIFO	AsyncNotificationQueue – backpressure via b
LinkedBlockingQueue	Queue	Yes	Optional	FIFO	Alternative unbounded async queue
PriorityBlockingQueue	Queue	Yes (take)	No	Priority	PriorityNotificationQueue – ordered by .priorit
ArrayDeque	Deque	No	No	LIFO or FIFO	OrderProcessingPipeline – addFirst/addLast/
LinkedList	Queue+Deque	No	No	FIFO / LIFO	NotificationDispatcher – O(1) head add/remov
ConcurrentLinkedQueue	Queue	No (poll null)	No	FIFO	Lock-free multi-producer/consumer queue
PriorityQueue	Queue	No	No	Min-heap	In-memory priority processing without blockin

```
// AsyncNotificationQueue – bounded with backpressure
BlockingQueue q = new ArrayBlockingQueue<>(500);
// producer blocks when full (natural backpressure)
q.put(task); // blocks if queue full
// consumer waits for work
NotificationTask t = q.poll(1, TimeUnit.SECONDS); // null if nothing arrives in 1s
// PriorityQueue – natural ordering via Comparable
PriorityQueue pq = new PriorityQueue<>(
Comparator.comparing(Notification::getPriority).reversed());
```

Deque as Stack & Queue

Operation	Queue (FIFO)	Stack (LIFO)	Deque method
Insert	offer(e) / add(e)	push(e)	addLast / addFirst
Remove	poll() / remove()	pop()	removeFirst / removeLast
Inspect	peek()	peek()	peekFirst / peekLast

01-06 **Collections Internals**

HashMap Bucket Architecture

HashMap<K,V> backing: Node<K,V>[] default cap=16 loadFactor=0.75

- Each bucket[i] = linked list of Node(key, value, hash, next)
- When bucket chain > 8 AND table.length >= 64 → convert to TreeNode (Red-Bla
  - TreeNode: O(log n) worst-case per bucket vs O(n) plain chain

put(key, value) steps:

1. hash(key) = key.hashCode() ^ (key.hashCode() >>> 16) — spread upper bit
2. i = hash & (n-1) — faster than modulo (n is always power of 2)
3. If bucket[i] empty → insert. Else traverse chain: hash match + equals ch

Resize: when size > capacity × 0.75 → double capacity, rehash all entries O

Parameter	Default	Meaning	Change When?
initialCapacity	16	Starting bucket array size	Set to expectedSize/0.75 + 1 to avoid resizes
loadFactor	0.75	Resize threshold = capacity × loadFactor	<0.75 wastes memory; >0.75 more collisions
treeify threshold	8	Bucket list converts to tree when ≥ 8 nodes	Internal; not configurable
untreeify threshold	6	Tree converts back to list when ≤ 6 nodes	Internal

ConcurrentHashMap Internals (Java 8+)

```
// No global lock. Each bin locked independently via CAS + synchronized(bin)
// Read path: volatile reads – zero locking for gets
// Write path: CAS for empty bins; synchronized(head) for occupied bins
// size(): uses LongAdder-based summing cells to avoid contention
```

■ ConcurrentHashMap does NOT allow null keys or values. A null return from get() is unambiguous: key not present.

ArrayList Internal Growth

Trigger	Action	New Capacity	Cost
add() when size == length	Arrays.copyOf to new array	oldCap × 1.5	O(n) copy – amortised O(1) per add
ensureCapacity(n)	Pre-allocate to n	n	Avoids multiple resizes for bulk inserts
trimToSize()	Shrink array to current size	size	Reduces memory footprint after bulk remove

01-07 Generics

Generic Classes in ShopVerse

Class	Signature	Constraint	Purpose
ApiResponse<T>	class ApiResponse<T>	Any T	Wraps every REST response: { data:T, status, mess
PagedResponse<T>	class PagedResponse<T extends Identifiable>	T must be Identifiable	Paged list with cursor metadata
GenericRepository<T,ID>	interface GenericRepository<T,ID>	None	findById, save, delete – all Spring Data repos extend
Result<T,E>	sealed interface Result<T, E extends AppException> permits Success, Failure	E must extend AppException	Success<T> | Failure<E> – no null returns

```
// ApiResponse usage
return ResponseEntity.ok(ApiResponse.success(orderResponse));
return ResponseEntity.badRequest().body(ApiResponse.error("Order not found",
ErrorCode.ORDER_NOT_FOUND));
```

Wildcard Summary – PECS Rule

PECS: Producer Extends, Consumer Super

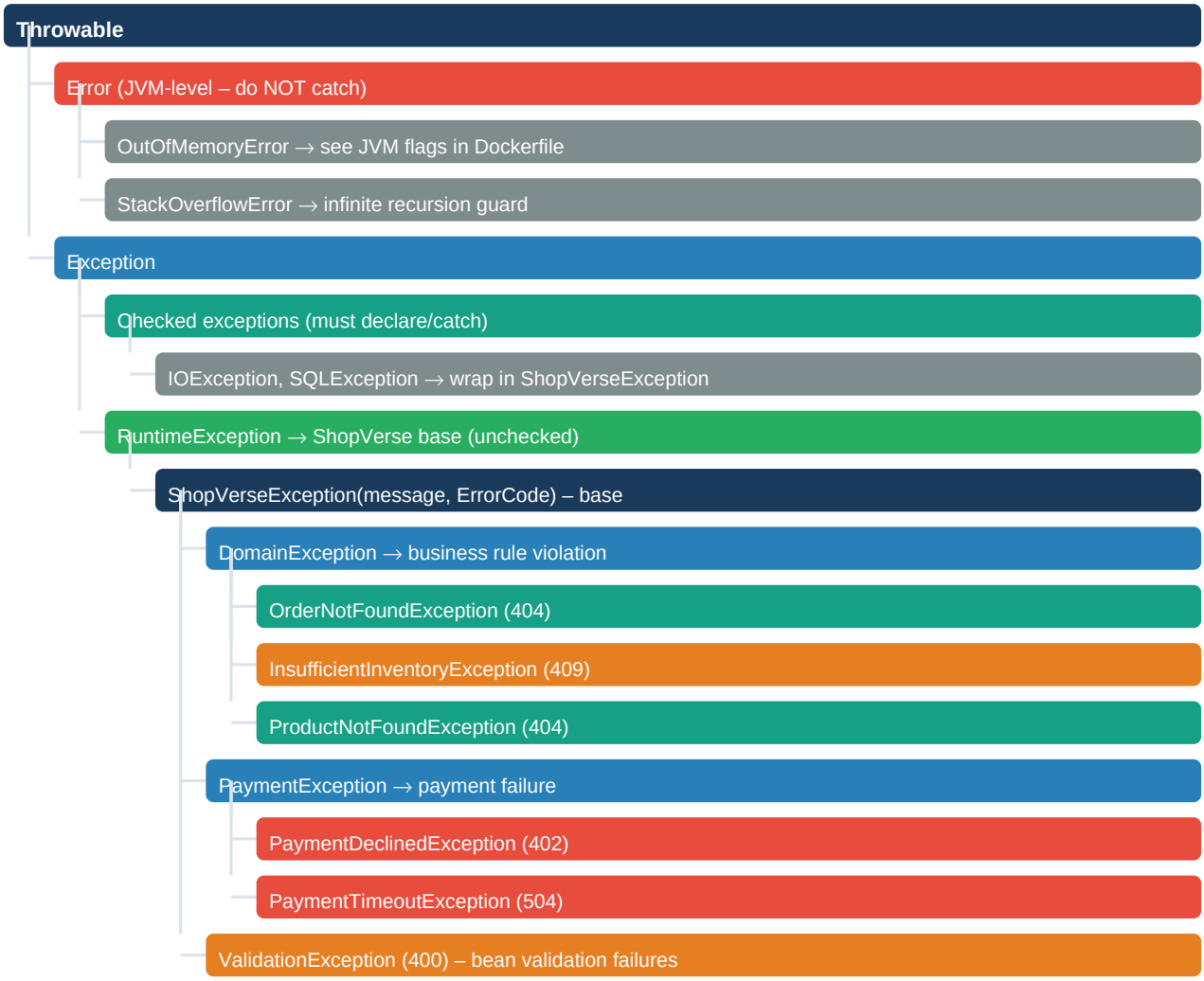
Wildcard	Meaning	Can Read As	Can Write	Rule
<?>	Any unknown type	Object	Cannot	Purely read-only; type doesn't matter
<? extends T>	T or subtype (upper bound)	T	Cannot	Producer – source of T items (read from)
<? super T>	T or supertype (lower bound)	Object only	T or subtype	Consumer – sink for T items (write into)

```
// PECS example
void copy(List src, List dst) {
for (Number n : src) dst.add(n); // read from Producer, write to Consumer
}
```

Type Erasure Effects

What you write	What JVM sees at runtime	Implication
List<String>	List (raw)	Cannot distinguish List<String> from List<Integer>
new T()	ILLEGAL	Use Class<T> token or Supplier<T> instead
instanceof List<String>	ILLEGAL	Use instanceof List<?> instead
T[] arr = new T[n]	ILLEGAL	Use (T[]) new Object[n] with unchecked cast
Overload foo(List<A>) + foo(List<B>)	CLASH	Same erasure – compiler error

ShopVerse Exception Hierarchy



Exception	HTTP Code	ErrorCode Enum	When Thrown
OrderNotFoundException	404 Not Found	ORDER_NOT_FOUND	findById returns empty Optional
InsufficientInventoryException	409 Conflict	INSUFFICIENT_INVENTORY	stockCount < requested qty
ProductNotFoundException	404 Not Found	PRODUCT_NOT_FOUND	Product does not exist
PaymentDeclinedException	402 Payment Required	PAYMENT_DECLINED	Gateway rejects charge
PaymentTimeoutException	504 Gateway Timeout	PAYMENT_TIMEOUT	Gateway call exceeds timeout
InvalidStateTransitionException	422 Unprocessable	INVALID_STATE_TRANSITION	Illegal order state change

```
// GlobalExceptionHandler – maps exceptions to HTTP responses
@RestControllerAdvice public class GlobalExceptionHandler {
@ExceptionHandler(OrderNotFoundException.class)
public ResponseEntity> handle(OrderNotFoundException ex) {
return ResponseEntity.status(NOT_FOUND)
.body(ApiResponse.error(ex.getMessage(), ex.getErrorCode()));
}
}
```

Stream Pipeline Operations

Category	Operations	Laziness	ShopVerse Example
Intermediate	filter, map, flatMap, distinct, sorted, peek, limit, skip	Lazy — no work until terminal	orders.stream().filter(o -> o.getStatus() != CANCELLED)
Terminal	collect, forEach, reduce, count, findFirst, toList()	Eager – triggers pipeline	.collect(Collectors.toUnmodifiableList())
Short-circuit terminal	findFirst, anyMatch, allMatch, noneMatch, limit	Stops early when result known	items.stream().anyMatch(i -> i.getPrice().compareTo(MI
Parallel	.parallelStream()	ForkJoin pool	Bulk price recalculation on catalog

```
// Revenue aggregation with reduce
BigDecimal total = order.getItems().stream()
.map(i -> i.getPrice().multiply(BigDecimal.valueOf(i.getQty())))
.reduce(BigDecimal.ZERO, BigDecimal::add);
// GroupingBy – orders per customer
Map> byCustomer = orders.stream()
.collect(Collectors.groupingBy(Order::getCustomerId));
// FlatMap – all items across all orders
List allItems = orders.stream().flatMap(o -> o.getItems().stream()).toList();
```

Optional Best Practices

Pattern	Code	Use When
orElseThrow	opt.orElseThrow(() -> new OrderNotFoundException(id))	Expected to exist; absence is an error
orElse	opt.orElse(Product.UNKNOWN)	Absence has a default value
orElseGet	opt.orElseGet(() -> computeDefault())	Default is expensive to compute – lazy
map + orElseThrow	opt.map(Mapper::toDto).orElseThrow(…)	Transform then fail if absent
ifPresent	opt.ifPresent(o -> audit(o))	Side effect only when present
filter	opt.filter(o -> o.isActive())	Additional predicate before use

■ Never use Optional.get() without isPresent(). Never return null from a method declared to return Optional.

Date/Time API

Class	Zone?	Mutable?	DB Column Type	ShopVerse Usage
Instant	No (always UTC)	No	TIMESTAMPTZ	createdAt, updatedAt on all entities
LocalDateTime	No	No	TIMESTAMP	Display formatting before zone is applied
ZonedDateTime	Yes	No	TIMESTAMPTZ	Customer-facing display using preferredZoneId
LocalDate	No	No	DATE	Report periods, date-only filters
Duration	N/A	No	BIGINT (ms)	TTL values, timeout configs
Period	N/A	No	VARCHAR	Human-readable date intervals

```
Instant stored = order.getCreatedAt(); // from DB (UTC)
ZoneId zone = ZoneId.of(customer.getPreferredZoneId());
ZonedDateTime display = stored.atZone(zone); // for API response
String formatted = DateTimeFormatter
.ofLocalizedDateTime(FormatStyle.MEDIUM).withLocale(locale).format(display);
```

Records (Java 16)

```
// Compiler generates: canonical constructor, getters, equals, hashCode, toString
public record PlaceOrderRequest(long customerId, List items, String promoCode) {
// Compact constructor – add validation
public PlaceOrderRequest {
Objects.requireNonNull(items, "items");
if (items.isEmpty()) throw new ValidationException("Order must have items");
}
}
// Value Object as record
public record Money(BigDecimal amount, String currency) {
public Money add(Money o) { return new Money(amount.add(o.amount), currency); }
}
```

Sealed Classes + Pattern Matching (Java 17/21)

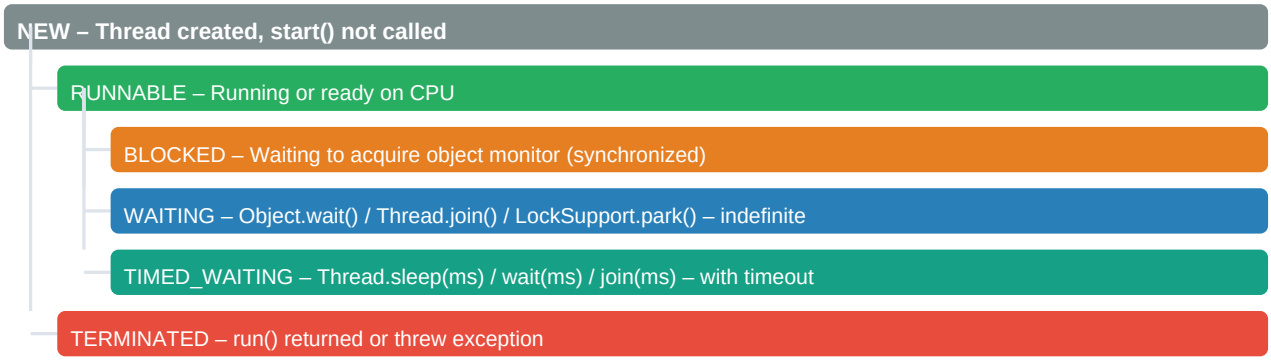
```
public sealed interface OrderEvent permits OrderPlacedEvent, OrderShippedEvent,
OrderCancelledEvent {}
public record OrderPlacedEvent(long orderId, Instant at) implements OrderEvent {}
public record OrderShippedEvent(long orderId, String tracking) implements OrderEvent {}
public record OrderCancelledEvent(long orderId, String reason) implements OrderEvent {}
// Exhaustive pattern switch – compiler validates all permitted types covered
String msg = switch (event) {
case OrderPlacedEvent e -> "Placed " + e.orderId() + " at " + e.at();
case OrderShippedEvent e -> "Shipped " + e.orderId() + " tracking " + e.tracking();
case OrderCancelledEvent e-> "Cancelled " + e.orderId() + ": " + e.reason();
};
```

Feature	Java Version	ShopVerse Class/Usage
Records	16 (final)	All DTOs: PlaceOrderRequest, OrderResponse, ProductSummary, Money, Address
Sealed classes	17 (final)	OrderEvent sealed interface; OrderStateMachine sealed state types
Pattern matching instanceof	16 (final)	Reduced casting in EventHandler, visitor code
Pattern switch (exhaustive)	21 (final)	EventHandler.handle(OrderEvent) – no default needed
Text blocks	15 (final)	Multi-line JPQL @Query annotations; JSON template strings
Switch expression	14 (final)	PricingStrategy selection on CustomerTier; NotificationType routing
Virtual Threads (Loom)	21 (final)	spring.threads.virtual.enabled=true for all HTTP request threads
SequencedCollection	21 (final)	Cart display, recently-viewed list (getFirst/getLast)
Structured Concurrency	21 (preview)	Future – scoped task management

```
// Virtual thread comparison
// Platform thread: ~1 MB stack; mapped 1:1 to OS thread; max ~10 000
// Virtual thread: ~few KB; M:N mapped to carrier threads; millions possible
// spring.threads.virtual.enabled=true → all @Async, WebMVC, scheduled use virtual threads
```

■ Virtual threads eliminate async programming complexity for JDBC workloads. Blocking a virtual thread just parks it; the carrier thread is reused immediately.

Thread State Diagram



ExecutorService Patterns

Pattern	Factory Method	Pool Size	Use Case
Fixed Thread Pool	Executors.newFixedThreadPool(n)	n threads always	CPU-bound tasks (n = #cores + 1)
Cached Thread Pool	Executors.newCachedThreadPool()	0 to ∞ (60s idle TTL)	Short-lived async tasks; spiky load
Single Thread Executor	Executors.newSingleThreadExecutor()	1 thread	Sequential task queue; in-order processing
Scheduled Pool	Executors.newScheduledThreadPool(n)	n threads	@Scheduled equivalent; periodic tasks
Work-Stealing Pool	Executors.newWorkStealingPool()	#CPU cores	Parallel stream tasks; fork-join workloads
Virtual Thread Executor	Executors.newVirtualThreadPerTaskExecutor()	Unlimited	I/O-heavy tasks; Java 21+

```
// NotificationThreadPool – named factory for diagnostic thread dumps
ExecutorService pool = new ThreadPoolExecutor(5, 20, 60L, SECONDS,
new ArrayBlockingQueue<>(100),
new ThreadFactoryBuilder().setNameFormat("notification-%d").build(),
new CallerRunsPolicy()); // backpressure: caller thread runs task when queue full
// OrderCompletionTask – Callable with timeout
Future f = pool.submit(() -> processOrder(orderId));
try { return f.get(30, SECONDS); }
catch (TimeoutException e) { f.cancel(true); throw new PaymentTimeoutException(); }
```

Synchronisation Mechanisms Comparison

Mechanism	Fairness	Condition Vars	Interruptible	Timed Try	ShopVerse
synchronized	No	wait/notify (single)	No	No	Low-level demo in InventoryRese
ReentrantLock	Optional (fair=true)	Multiple Conditions	Yes	Yes (tryLock)	Replacement – InventoryReserve
ReentrantReadWriteLock	Optional	Via writeLock	Yes	Yes	Heavy-read data – catalog cache
StampedLock	No	No	Yes	Yes (tryOptimisticRead)	Highest-perf alternative to RWLoc

```
// ReentrantLock guarantees unlock even on exception
lock.lock(); try { doWork(); } finally { lock.unlock(); }
// tryLock with timeout – avoids indefinite blocking
if (lock.tryLock(500, MILLISECONDS)) { try { ... } finally { lock.unlock(); } }
```

Atomic Classes

Class	CAS Operation	ShopVerse Usage
AtomicLong	compareAndSet(expect, update)	OrderIdGenerator.next() – lock-free monotonic IDs
AtomicInteger	incrementAndGet(), decrementAndGet()	Stock count CAS in optimistic reserve path
AtomicBoolean	compareAndSet(false, true)	One-time initialisation guard; shutdown flag
AtomicReference<V>	compareAndSet(oldRef, newRef)	Hot-swap config object without locking
LongAdder	add(long) / sum()	High-contention counter (ConcurrentHashMap.size uses this)

```
private final AtomicLong counter = new AtomicLong(0);
public long next() { return counter.incrementAndGet(); } // lock-free, thread-safe
```

Synchronisation Utilities

Class	Mechanism	Reusable?	ShopVerse Class	Purpose
CountDownLatch	countDown() / await()	No (one-shot)	ApplicationStartupValidator	Wait for DB+Redis+Kafka ready
CyclicBarrier	await() – all meet barrier	Yes (reset)	—	Multi-phase parallel batch
Semaphore	acquire() / release()	Yes	FlashSaleService	Limit concurrent flash sales to 10
Exchanger	exchange(V)	Yes	—	Pair of threads swap objects
Phaser	register/arrive/awaitAdvance	Yes (dynamic)	—	Dynamic participant count per phase

```
// CountDownLatch – startup gate
CountDownLatch ready = new CountDownLatch(3);
checkDb().thenRun(ready::countDown);
checkRedis().thenRun(ready::countDown);
checkKafka().thenRun(ready::countDown);
ready.await(60, SECONDS); // block until all signal or timeout
// Semaphore – concurrency limiter
private final Semaphore slots = new Semaphore(10, true); // fair
slots.acquire(); try { processFlashSale(); } finally { slots.release(); }
```

CompletableFuture Chaining

```
CompletableFuture.supplyAsync(() -> fetchProduct(id), executor)
.thenApply(product -> enrichWithReviews(product))
.thenCompose(enriched -> calculatePrice(enriched))
.exceptionally(ex -> { log.error("Failed", ex); return defaultProduct(); })
.thenAccept(result -> cache.put(id, result));
```

Method	Input→Output	Use
thenApply(fn)	T → U	Sync transform (map equivalent)
thenCompose(fn)	T → CompletableFuture<U>	Async chain (flatMap equivalent)
thenCombine(cf, fn)	T + U → V	Merge two independent futures
allOf(cf…)	N futures → void	Wait for all to complete
anyOf(cf…)	N futures → first	Return first completed
exceptionally(fn)	Throwable → T	Error recovery / fallback

01-13 JVM Internals

JVM Memory Layout

GC Algorithms Comparison

GC	Flag	Pause Pattern	Heap Size	ShopVerse Choice
Serial	UseSerialGC	Long STW; single thread	< 100 MB	No – development only
Parallel	UseParallelGC	Medium STW; multi-thread	< 4 GB	No – batch jobs only
G1GC	UseG1GC	Short predictable STW + concurrent	4–32 GB	YES – default for ShopVerse
ZGC	UseZGC	< 1ms; concurrent everywhere	Any (TB scale)	Optional for ultra-low latency
Shenandoah	UseShenandoahGC	< 1ms; concurrent	Any	Red Hat JDK alternative

ShopVerse Dockerfile JVM Flags

Flag	Value	Purpose
-XX:+UseG1GC	on	Garbage-First GC – predictable pause targeting
-Xms	-Xms512m	Initial heap – pre-allocate to avoid early resizes
-Xmx	-Xmx1g	Max heap – prevent OOM on container
-XX:MaxGCPauseMillis	200	G1 targets ≤ 200ms STW pause
-XX:+HeapDumpOnOutOfMemoryError	on	Auto heap dump on OOM for post-mortem analysis
-XX:HeapDumpPath	/dumps/	Container volume mount for heap dump files
-XX:+UseStringDeduplication	on	G1 deduplicates equal char[] arrays – saves ~10% heap

Flag	Value	Purpose
-XX:+PrintGCDetails (dev)	on	GC log verbosity for profiling

```
// JvmMetricsEndpoint – custom Actuator endpoint
@Endpoint(id="jvm-metrics") public class JvmMetricsEndpoint {
@ReadOperation public Map get() {
MemoryMXBean m = ManagementFactory.getMemoryMXBean();
return Map.of("heapUsedMB", m.getHeapMemoryUsage().getUsed()/1_048_576,
"heapMaxMB", m.getHeapMemoryUsage().getMax()/1_048_576,
"gcPausesMs", ManagementFactory.getGarbageCollectorMXBeans()
.stream().mapToLong(GarbageCollectorMXBean::getCollectionTime).sum());
}
}
```

BP Best Practices & Anti-Patterns

■ Do

- ✓ Use long for all entity IDs – int overflows silently at ~2.1 billion.
- ✓ BigDecimal(String) for money – never BigDecimal(double).
- ✓ Store secrets as char[] and zero them after use.
- ✓ Return Optional from repository methods – no null returns.
- ✓ Use Collections.unmodifiableList() when returning internal collections.
- ✓ Prefer EnumSet/EnumMap for enum-keyed collections (faster + less memory).
- ✓ Restore interrupt status in catch(InterruptedException): Thread.currentThread().interrupt().
- ✓ Use Instant for storage; ZonedDateTime only for customer display.
- ✓ Pre-size collections when final size is known: new ArrayList<>(expectedSize).
- ✓ Use Executors.newVirtualThreadPerTaskExecutor() for IO-heavy async tasks in Java 21.
- ✓ Prefer computeIfAbsent over manual null-check + put in concurrent maps.
- ✓ Use text blocks for multi-line SQL/JSON strings – improves readability significantly.

■ Anti-Patterns to Avoid

- ■ double/float for money – 0.1 + 0.2 ≠ 0.3.
- ■ Null returns from service methods – breaks callers silently.
- ■ Raw types (List instead of List) – bypasses compiler checks.
- ■ Catching Exception/Throwable broadly and swallowing it (empty catch block).
- ■ Thread.stop() / Thread.suspend() – deprecated, unsafe, may corrupt state.
- ■ Shared mutable state without synchronisation – data race / visibility bugs.
- ■ Unbounded thread creation (new Thread() per request) – OutOfMemoryError under load.
- ■ HashMap in concurrent code without external synchronisation.
- ■ == for String comparison – compares reference, not value.
- ■ Ignoring Future.get() result – unchecked exceptions silently discarded.
- ■ Creating new BigDecimal(0.1) from double literal – still imprecise.
- ■ Using StringBuffer where StringBuilder suffices – unnecessary synchronisation.

QR Quick Reference – Sub-chapter → ShopVerse Class Map

Sub-Chapter	Key ShopVerse Class(es)	Package
01-01 Primitives	MoneyConverter, JwtProperties	infrastructure.converter / config
01-02 List	OrderService (ArrayList), NotificationDispatcher (LinkedList)	order.service / notification
01-03 Set	ProductTag (HashSet), CategoryService (TreeSet), OrderValidator (EnumSet)	catalog.service
01-04 Map	ShoppingCart (LinkedHashMap), RateLimiter (ConcurrentHashMap), LocaleConfig (EnumMap)	cart / config
01-05 Queue	AsyncNotificationQueue (ArrayBlockingQueue), OrderProcessingPipeline (ArrayDeque)	notification / order.service
01-06 Internals	CartService.java (comment block)	cart.service
01-07 Generics	ApiResponse<T>, PagedResponse<T>, GenericRepository<T,ID>, BaseEntity	repository / domain
01-08 Exceptions	ShopVerseException hierarchy, ErrorCode, GlobalExceptionHandler	exception / web
01-09 Java 8	OrderService (streams), OrderRepo (Optional), NotificationService (CF)	order.service / repository / notification
01-10 Java 9-21	PlaceOrderRequest (record), OrderEvent (sealed), EventHandler (switch)	dto / event / handler
01-11 Multithreading	NotificationThreadPool, InventoryReserveService, CleanupTask	notification / inventory / scheduler
01-12 j.u.concurrent	OrderIdGenerator, ApplicationStartupValidator, FlashSaleService	order.service / config
01-13 JVM	Dockerfile (flags), JvmMetricsEndpoint, application.yaml	root / actuator / resources

Interview Quick-Check

Question	Concise Answer
Why long for IDs?	int overflows at ~2.1B. A busy system can hit this in days.
Why BigDecimal for money?	IEEE-754 binary floats cannot represent 0.1 exactly. BigDecimal is exact decimal arithmetic.
HashMap vs CHM?	HashMap is unsynchronised. ConcurrentHashMap uses CAS per bin – concurrent reads are lock-fr
Type erasure?	Generics are compile-time only. List<String> → List at runtime. Cannot instanceof List<String>.
Virtual thread vs platform?	VT: ~KB, millions possible, park on blocking. PT: ~MB, OS-mapped, blocks carrier.
Optional.get()?	Dangerous – throws NoSuchElementException. Use orElseThrow/orElse/map.
Latch vs Semaphore?	Latch: one-shot gate (wait for N events). Semaphore: reusable concurrency limit.
G1GC pause target?	MaxGCPauseMillis=200 tells G1 to aim for ≤ 200ms STW, but it is a soft target.